AULA 04 — BASES NUMÉRICAS E REPRESENTAÇÃO COMPUTACIONAL

Aluna: *Victoria Antonia Brito Barbosa*
Turma: *NB*
Disciplina: *Cálculo Numérico*
Ambiente utilizado: *Python / Google Colab*

1. Introdução

Nesta aula, foram estudadas diferentes formas de representação dos números nos sistemas computacionais, com foco nos sistemas de numeração, nas conversões entre bases e na maneira como os computadores armazenam e processam valores inteiros e fracionários.

O conteúdo também abordou as limitações da representação numérica nos computadores, apresentando conceitos como complemento de dois, overflow, ponto fixo, ponto flutuante e erros de arredondamento. Por meio de exemplos e experimentos em Python, foi possível observar como essas limitações influenciam os resultados de operações matemáticas e a precisão dos cálculos.

2. Objetivos da aula

- Compreender os sistemas de numeração e suas aplicações na computação.
- Realizar conversões entre as bases decimal, binária, octal e hexadecimal.
- Entender como os computadores representam números positivos, negativos e fracionários.
- Conhecer as limitações da representação de números com quantidade finita de bits.
- Identificar situações de overflow e erros de arredondamento.
- Compreender a diferença entre ponto fixo e ponto flutuante.
- Utilizar Python para realizar conversões, testar operações e analisar erros numéricos.

3. Conteúdos estudados

3.1 Sistema hexadecimal

Estudei o sistema hexadecimal, que utiliza a base 16 e os símbolos de 0 a 9 e de A a F, sendo as letras equivalentes aos valores de 10 a 15. Também aprendi que cada algarismo hexadecimal corresponde a quatro bits, facilitando a conversão entre hexadecimal e binário.

Foram apresentados exemplos de conversão entre hexadecimal, decimal e binário, além de aplicações desse sistema na computação, como endereços de memória e representação de cores em HTML.

3.2 Representação de cores em hexadecimal

Aprendi que as cores em HTML podem ser representadas pelo padrão #RRGGBB, em que cada par de caracteres indica a intensidade de vermelho (R), verde (G) e azul (B). Cada componente varia de 00 a FF, correspondendo aos valores decimais de 0 a 255.

3.3 Conversão de decimal para outras bases

Estudei o método das divisões sucessivas para converter números decimais em outras bases. Nesse processo, o número é dividido repetidamente pela base desejada, e os restos são utilizados na representação final, sendo lidos de baixo para cima.

Também acompanhei a implementação desse método em Python, utilizando estruturas como while, append(), divmod() e join().

3.4 Operações aritméticas em outras bases

Aprendi que operações como adição também podem ser realizadas diretamente em outras bases numéricas. O processo é semelhante ao sistema decimal, mas os transportes acontecem quando a soma atinge o valor da base utilizada.

Foram apresentados exemplos de adição nas bases 2 e 4, com conferência dos resultados por meio da conversão para decimal.

3.5 Representação de números negativos

Estudei três métodos utilizados para representar números negativos em computadores:

- **Sinal e magnitude:** utiliza o primeiro bit para indicar o sinal e os demais para representar o valor absoluto.
- **Complemento de 1:** representa o número negativo invertendo todos os bits da representação positiva.
- **Complemento de 2:** inverte os bits e soma 1 ao resultado.

Também aprendi a converter números em complemento de 2 para decimal, realizar operações com números negativos e identificar overflow. Foi apresentada a extensão de sinal, utilizada para aumentar a quantidade de bits sem alterar o valor representado.

3.6 Conversão fracionária em outras bases

Aprendi que a conversão de frações decimais para outras bases é realizada por multiplicações sucessivas da parte fracionária pela base desejada. A parte inteira de cada multiplicação forma os algarismos da representação.

Também compreendi que nem todas as frações possuem representação finita em qualquer base. Isso depende dos fatores primos do denominador depois de simplificado.

3.7 Erros de representação numérica

Estudei por que determinados números, como 0,1, podem causar diferenças nos resultados das operações computacionais. Embora possuam representação finita em decimal, alguns números apresentam representação periódica em binário.

Como o computador possui uma quantidade limitada de bits para armazenar esses valores, precisa utilizar aproximações, o que pode provocar erros de arredondamento.

3.8 Representação com quantidade limitada de bits

Aprendi que a quantidade de bits determina o intervalo de valores que pode ser representado por um número inteiro sem sinal. Com n bits, os valores possíveis vão de zero até $2^n - 1$.

Também estudei o conceito de overflow, que acontece quando o resultado de uma operação ultrapassa o limite de representação disponível.

3.9 Representação em ponto fixo

Estudei a representação em ponto fixo, na qual uma quantidade definida de bits é utilizada para representar a parte fracionária de um número.

Compreendi que aumentar a quantidade de bits fracionários reduz o intervalo entre os valores representáveis e permite obter aproximações mais precisas, diminuindo o erro de quantização.

3.10 Introdução ao ponto flutuante

Aprendi que o ponto flutuante funciona de maneira semelhante à notação científica, utilizando o sinal, a significância e o expoente para representar os números.

Também conheci o epsilon da máquina, que representa a distância entre 1 e o próximo número de ponto flutuante representável. Esse conceito ajuda a compreender a precisão limitada dos computadores.

3.11 Comparação de números de ponto flutuante

Compreendi que testar a igualdade exata entre números de ponto flutuante pode gerar resultados inesperados devido aos erros de arredondamento.

Por isso, aprendi que é mais adequado utilizar uma tolerância para verificar se dois valores são suficientemente próximos. Em Python, essa comparação pode ser realizada com a função math.isclose().

3.12 Experimentos sobre erros numéricos

Por meio dos experimentos em Python, observei como o aumento da quantidade de bits pode melhorar a aproximação de um número e reduzir o erro absoluto.

Também estudei a acumulação de pequenos erros, percebendo que diferenças muito pequenas podem se tornar relevantes quando uma operação é repetida várias vezes. A função math.fsum() foi apresentada como uma alternativa para realizar somas com maior precisão.

4. Atividades realizadas

Atividade 1 - Conversão de números para decimal

Foram realizadas conversões de números binários, octais e hexadecimais para a base decimal.

Resultados:

- $11001_2 = 25_{10}$
- $237_8 = 159_{10}$
- $3A_{16} = 58_{10}$

A atividade também apresentou a utilização da função int() em Python para verificar as conversões.

Atividade 2 - Conversão de números decimais para outras bases

Foram realizadas conversões dos números 25, 100 e 255 para as bases 2, 8 e 16.

Decimal	Binário	Octal	Hexadecimal
25	11001	31	19
100	1100100	144	64
255	11111111	377	FF

Também foi estudada a estimativa da quantidade de algarismos necessária para representar um número em determinada base.

Atividade 3 - Números negativos utilizando 8 bits

1. Represente -25 em sinal e magnitude.

RESPOSTA: 10011001

2. Represente -25 em complemento de 1.

RESPOSTA: 11100110

3. Represente -25 em complemento de 2.

RESPOSTA: 11100111

4. Converta 11100110_2 de complemento de 2 para decimal.

RESPOSTA: -26

5. Determine se 90 + 60 provoca overflow.

RESPOSTA: Sim. A soma resulta em 150, ultrapassando o limite máximo de 127 para números de 8 bits em complemento de 2.

6. Explique por que o complemento de 2 possui uma única representação para zero.

RESPOSTA: Porque o zero é representado apenas por 00000000, não existindo uma representação separada para o zero negativo.

5. Utilização do Python

Durante a aula, utilizei o Python no Google Colab para acompanhar os exemplos e realizar experimentos relacionados aos conteúdos estudados.

Foram utilizadas funções e recursos como:

- int() para conversões entre bases.
- divmod() para divisões sucessivas.
- append() e join() para organizar os algarismos das conversões.
- while e for para repetir operações.
- format() para controlar a exibição de casas decimais.
- as_integer_ratio() para visualizar a fração armazenada por um número float.
- math.isclose() para comparar números utilizando tolerância.
- math.fsum() para realizar somas com maior precisão.
- sys.float_info para consultar informações sobre a representação de ponto flutuante.
- matplotlib para visualizar graficamente a redução dos erros de aproximação.

6. Conclusão

Nesta aula, ampliei meus conhecimentos sobre os sistemas de numeração e a maneira como os computadores representam e armazenam informações numéricas. Ao longo do conteúdo, estudei diferentes bases, realizei conversões entre elas e compreendi melhor a utilização dos sistemas binário e hexadecimal na computação.

Também aprendi que existem diferentes maneiras de representar números negativos, como sinal e magnitude, complemento de 1 e complemento de 2, cada uma com suas características e limitações. Consegui entender como a quantidade de bits influencia os valores que podem ser armazenados e por que situações de overflow podem acontecer quando um resultado ultrapassa o intervalo permitido.

Outro assunto importante foi a representação de números fracionários. Compreendi que nem todos os números possuem uma representação exata em binário e que isso explica algumas diferenças encontradas nos resultados de operações realizadas pelo computador. Os conceitos de ponto fixo e ponto flutuante me ajudaram a entender melhor como os números são armazenados, como a precisão é limitada e por que os erros de arredondamento acontecem.

Os exemplos e experimentos realizados em Python foram importantes para visualizar esses conceitos na prática. Consegui observar como o aumento da quantidade de bits melhora determinadas aproximações, como pequenos erros podem se acumular e como a utilização de tolerâncias pode tornar as comparações numéricas mais adequadas.

De modo geral, considero que esta aula foi importante para minha formação em Cálculo Numérico, pois me permitiu compreender que os computadores não trabalham necessariamente com valores reais exatos, mas com representações limitadas desses valores. Também percebi a importância de conhecer essas limitações para interpretar corretamente os resultados de cálculos e desenvolver programas que apresentem maior precisão e confiabilidade.